Scaling Neural Network Verification with Tensor Parallelism and Fully Sharded Data Parallelism

Sergei Vorobyov
Lomonosov Moscow State University
sergei.vorobyov01@gmail.com

Evgeny Ilyushin
Lomonosov Moscow State University

Abstract

Formal neural network verification—proving that a network satisfies safety properties for *all* inputs in a specified domain—is bounded in practice by GPU memory: bound-propagation algorithms (IBP, CROWN, α-CROWN) require weight and relaxation-coefficient matrices to reside entirely on one accelerator. We adapt two parallelism techniques originally developed for large-scale model training to the `auto_LiRPA` / α, β-CROWN verification framework. **Tensor Parallelism (TP)** shards both weight and A-matrices across GPUs, achieving $\approx 2\times$ peak-memory reduction at $P=2$; soundness is confirmed on VNN-COMP 2022 MNIST-FC benchmarks, though bound tightness degrades with the number of sharded zones due to forced IBP substitution inside sharded zones. **Fully Sharded Data Parallelism (FSDP)** shards only weight matrices with a per-layer `AllGather`, producing bounds that are *bitwise identical* to the single-GPU baseline: baseline memory drops by 80–90%, peak memory by 34–39% on wide MLPs. FSDP integrates cleanly with complete verification (β-CROWN + Branch-and-Bound) and with convolutional layers (`BoundConv`); a complete *unsat* result is obtained for CIFAR-100 ResNet-large (VNN-COMP 2024) under FSDP. Across all experiments the memory bottleneck in α-CROWN+BaB mode proves to be per-neuron alpha tensors, not weight matrices, pointing to the key direction for future work.

1 Introduction

Neural networks are increasingly deployed in safety-critical contexts where errors are intolerable: autonomous vehicle control systems, avionics software, and medical diagnostic algorithms. Standard testing methods are fundamentally insufficient in such settings: a test verifies a specific input vector, not the entire set of admissible inputs [13]. Formal verification addresses exactly this gap—it provides a mathematical proof that the network behaves correctly for *every* input in a specified region.

1.1 Formal Verification Problem

Let a neural network define a function $f : \mathbb{R}^n \to \mathbb{R}^m$. The verification problem is to decide whether, for all inputs in an admissible set $\mathcal{X} \subseteq \mathbb{R}^n$, an output property ψ holds:

$$\forall \mathbf{x} \in \mathcal{X}, \quad \psi(f(\mathbf{x})).$$

In practice, verifiers solve the dual problem—they search for a *counterexample* $\mathbf{x} \in \mathcal{X}$ violating ψ:

$$\exists \mathbf{x} \in \mathcal{X} : \neg\psi(f(\mathbf{x})).$$

If the query is unsatisfiable (UNSAT), the property is proved; if satisfiable (SAT), a counterexample is returned [16].

The most widely studied property is *local robustness*: for a sample $\mathbf{x}_0$ of correct class c, the classifier must assign class c to all inputs within an ε-ball, $\|\mathbf{x} - \mathbf{x}_0\|_p \leq \varepsilon$ [15]. Beyond image classifiers, functional specifications over physical variables are used in control tasks such as ACAS Xu [11].

1.2 The VNN-LIB Standard and VNN-COMP

Prior to 2020, each verifier used its own format for network and property descriptions. The VNN-LIB standard [10] unified the specification language based on SMT-LIB: input/output variables and linear constraints over them with UNSAT/SAT semantics. Together with ONNX for weight storage, VNN-LIB forms the infrastructure for the annual VNN-COMP competition [3, 4, 7], where benchmarks and tools are developed independently.

1.3 Computational Complexity

Verification is NP-complete for single-hidden-layer ReLU networks, as 3-SAT reduces polynomially to it [6]. For smooth activations (Sigmoid, Tanh, GELU) the complexity is $\exists\mathbb{R}$-complete [9]. For certain classes of recurrent networks, reachability is undecidable [21]. In practice this means a *complete* verifier must use worst-case exponential Branch-and-Bound, while an *incomplete* verifier replaces it with a polynomial relaxation at the cost of losing completeness guarantees.

1.4 Memory Bottleneck

The dominant practical approach to verification is *bound propagation*. α, β-CROWN [28]—winner of VNN-COMP 2021–2024—runs entirely on GPU and builds linear over-approximations of the network output set. Its bottleneck is memory: weight matrices and linear-relaxation A-matrices must fit on a single accelerator. A single hidden layer of width $h{=}262\,144$ with input dimension $4\,096$ already requires $\approx$4 GB in float32 for one batch—exceeding a single A40 GPU.

This paper investigates how *parameter-sharding* strategies developed for large-scale model training can remove that constraint, and experimentally characterises the accuracy–memory trade-offs each approach entails.

Contributions

1. We implement **Tensor Parallelism** for `auto_LiRPA` via custom `BoundLinearTP_Col/Row` operators, achieve $\approx 2\times$ peak-memory reduction at $P{=}2$, and formally analyse the bound-tightness degradation caused by IBP substitution inside sharded zones.
2. We implement **FSDP** for bound propagation in $\approx$100 lines, producing bitwise-identical bounds with 34–39% peak and 80–90% baseline memory savings on wide MLPs.
3. We integrate FSDP with **complete verification** (β-CROWN+BaB), introduce a `force_synchronous` flag for fair comparison, and extend sharding to **convolutional layers** (`BoundConv`).
4. We identify **per-neuron alpha tensors**—not weights—as the true memory bottleneck in BaB mode, shifting the focus for future multi-GPU verification research.

2 Verification Algorithms

2.1 Hierarchy of Bound-Propagation Methods

Bound-propagation algorithms compute, for each network output, an interval guaranteed to contain the true value for any admissible input. Three levels of precision and cost are distinguished.

IBP (Interval Bound Propagation) [27] propagates intervals $[\mathbf{l}, \mathbf{u}]$ layer by layer. For a linear layer $\mathbf{y} = W\mathbf{x} + \mathbf{b}$:

$$\mathbf{l}_y = W^+\mathbf{l}_x + W^-\mathbf{u}_x + \mathbf{b}, \qquad \mathbf{u}_y = W^+\mathbf{u}_x + W^-\mathbf{l}_x + \mathbf{b},$$

where $W^+ = \max(W, 0)$, $W^- = \min(W, 0)$ elementwise. IBP is cheap ($O(n)$ per layer) but over-approximates rapidly with depth.

CROWN [27] computes tight linear bounds via a backward pass analogous to backpropagation. For a ReLU neuron with pre-activation $x \in [l, u]$, $l < 0 < u$, a linear relaxation is constructed: $\lambda_l x + b_l \leq \mathrm{ReLU}(x) \leq \frac{u}{u-l}(x - l)$. The backward pass links the output to the input through a chain of linear inequalities, building the A-matrix:

$$A = \Lambda_L W_L \cdots \Lambda_1 W_1, \qquad \ell \;\leq\; A\mathbf{x} + \mathbf{b} \;\leq\; u, \tag{1}$$

where $\Lambda_i = \mathrm{diag}(\lambda_i)$ are the per-neuron relaxation slope matrices. The key property is that A is a *global* linear function of the original input $\mathbf{x}$, preserving cross-variable dependencies across all layers—which makes CROWN substantially tighter than IBP.

α-CROWN [28] promotes the fixed slopes λ_i to per-neuron optimisable parameters $\alpha_i \in [0, 1]$. Gradient ascent over α tightens the lower bound (or lowers the upper bound) without any branching, yielding the tightest incomplete bounds possible.

2.1.1 Incomplete vs. Complete Verification

IBP, CROWN, and α-CROWN are *incomplete* methods: they construct a sound over-approximation (lower bounds never exceed true minima; upper bounds never fall below true maxima). When the resulting bounds are tight enough to confirm the property, verification succeeds; otherwise the outcome is indeterminate. No false positive can occur.

Complete verification [25] guarantees a definitive answer via **Branch-and-Bound (BaB)**: when bounds are insufficient, the input space is split into sub-regions (e.g. by fixing a ReLU state $x \geq 0$ or $x < 0$), and bounds are recomputed for each sub-region. The process repeats recursively until an answer is obtained or a timeout is reached.

2.2 Alpha-Beta-CROWN

α, β-CROWN [28] unifies all methods above. For incomplete verification it uses α-CROWN; when that does not yield tight enough bounds it invokes BaB. The key innovation of β-CROWN [25] is encoding ReLU-split constraints directly into the bound-propagation pass via Lagrangian dual variables β, avoiding the need to solve a linear program at each BaB leaf. As a result, bounds for thousands of sub-problems are evaluated in parallel on GPU as batched matrix multiplications, and

α/β parameters are jointly optimised by gradient descent. The cutting-plane extension BICCOS [22] generates constraints from logical implications in the BaB tree, tightening root-node bounds and reducing branching depth.

2.3 Other Verifiers

VeriNet uses Symbolic Interval Propagation (SIP): instead of linear inequalities, dependencies are maintained as symbolic expressions throughout the network. It is particularly effective for input splitting—a strategy suited to low-dimensional control tasks such as ACAS Xu [17].

nnenum [8] (winner of VNN-COMP 2020) dynamically switches between levels of set-representation precision—from fast but coarse zonotopes to accurate but expensive ImageStars—depending on current sub-problem complexity.

Marabou 2.0 [5] transforms the combinatorial search into continuous optimisation via the DeepSoI procedure (Sum of Infeasibilities), a continuous measure of total ReLU-constraint violation that can be minimised by gradient methods.

NeuralSAT [12] adapts CDCL (Conflict-Driven Clause Learning) from SAT solving: infeasible neuron-state combinations are recorded as nogoods, pruning entire subtrees in subsequent search.

2.4 The `auto_LiRPA` Library

All experiments in this work are built on `auto_LiRPA` [26], the library underlying α, β-CROWN. Bound-propagation operations are described as nodes of a PyTorch computational graph; the library automatically derives upper and lower bounds for any intermediate node of an arbitrary architecture (MLP, CNN, Transformer).

Critically, `auto_LiRPA` supports *batch BaB*: instead of processing BaB branches sequentially, it batches up to hundreds of thousands of unsolved sub-problems and computes bounds for all of them in a single GPU pass. This makes any modification to the library non-trivial: it must preserve batched processing and remain compatible with the PyTorch computational graph.

3 Parallel Bound Propagation

3.1 Parallelism Primitives

We briefly define the two strategies we adapt; readers familiar with distributed training may skip to Section 3.2.

Tensor Parallelism (TP) [23] splits matrix multiplications *within* a single layer across P GPUs. *Column-parallel*: weight $W \in \mathbb{R}^{k\times n}$ is split along the output dimension into shards $W^{(r)} \in \mathbb{R}^{k\times n/P}$; each GPU r computes a partial output $\mathbf{y}^{(r)} = W^{(r)\top}\mathbf{x}$ from a replicated input. *Row-parallel*: weight is split along the input dimension; each GPU multiplies its shard against its portion of the input and partial results are summed via `AllReduce`: $\mathbf{y} = \sum_r W^{(r)}\mathbf{x}^{(r)}$. A Column–Row pair requires exactly one `AllReduce` at the Row output.

Fully Sharded Data Parallelism (FSDP) [20] shards each parameter tensor equally across P GPUs. Before executing a layer, an `AllGather` assembles the full parameter from shards; immediately

after the layer finishes, the full tensor is freed and only the shard is retained. The net effect: at most one full weight matrix is in GPU memory at any instant.

Pipeline Parallelism (PP) [14] assigns consecutive groups of layers to different GPUs. As argued in Section 3.2, PP is incompatible with CROWN backward and is not pursued.

3.2 Why TP and FSDP, Not PP

The CROWN backward pass builds the A-matrix $A = \Lambda_L W_L \cdots \Lambda_1 W_1$ in one traversal through *all* layers (Equation 1). Under PP, each GPU holds only its stage; a complete CROWN backward would require transmitting weight matrices between GPU stages at every step—equivalent to TP with worse communication patterns. TP and FSDP both allow the full backward pass to run on every GPU, differing only in *what* is sharded: TP shards the computation itself, FSDP shards only the storage.

3.3 Tensor Parallelism for Verification

3.3.1 Implementation

We register two custom operators in `auto_LiRPA`: `BoundLinearTP_Col` (Column-parallel) and `BoundLinearTP_Row` (Row-parallel), both inheriting from `BoundLinear`. Each adds an `AllReduce` to `bound_backward` and registers a custom ONNX symbolic method so that PyTorch JIT tracing constructs the computational graph *without* invoking the collective—which would raise a `Tried to trace ProcessGroup` error.

The function `tp_shard_bounded_module` takes any `BoundedModule` (including ONNX-loaded models) and automatically replaces alternating `BoundLinear` pairs with TP variants, shards the weight matrices, and labels each Column–Row subgraph as a *sharded zone* via BFS traversal for correct intermediate-bound handling.

3.3.2 Bound-Tightness Degradation

TP shards both W and the A-matrices, so the CROWN backward step $A \leftarrow A \cdot \Lambda_i W_i$ can be computed locally per GPU for simple Column–Row pairs with no intermediate ReLU—only one `AllReduce` is needed to recover the correct result.

However, when ReLU activations appear *inside* a sharded zone (between Column and Row layers, as in typical networks deeper than two layers), CROWN needs *intermediate bounds* for those neurons to choose the relaxation slopes λ_i. Computing them exactly would require a separate CROWN backward with inter-GPU communication at each step, negating the memory savings. The only tractable alternative is IBP for those intermediate bounds.

IBP does not track cross-variable correlations: it treats each interval component independently, causing the *wrapping effect* [18]—bound widths grow as $O(\rho^k)$ with zone depth k, where ρ is the spectral radius of the elementwise-absolute weight matrix $|W|$.

Illustrative example. Consider $\mathbf{x} = (x_1, x_2)^\top$ with $x_1, x_2 \in [-1, 1]$ and $W = \left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$. CROWN: $z_1 = y_1 + y_2 = 2x_1 \in [-2, 2]$ (exact). IBP: after the first layer $y_1, y_2 \in [-2, 2]$; treating them as *independent*, $z_1 \in [-4, 4]$—twice the true width.

Experimentally (Table 2), a 256×2 model with one sharded zone (no intermediate ReLU) achieves machine-precision agreement ($\sim 10^{-8}$); a 256×4 model with two zones deviates by ~ 2.5; a 256×6 model with three zones deviates by ~ 750. Soundness (one-sided over-approximation) is preserved in all cases.

TP is therefore practical for shallow models (one or two sharded zones) and offers the highest peak-memory reduction. For deep models it is outperformed by FSDP in bound quality.

3.4 Fully Sharded Data Parallelism for Verification

3.4.1 Implementation

`fsdp_shard_bounded_module` traverses the `BoundedModule` graph and replaces each `BoundParams` tensor with a shard along `dim=0` (output channels): $W^{(r)} \in \mathbb{R}^{k/P \times n}$ for `BoundLinear` and $W^{(r)} \in \mathbb{R}^{C_{\text{out}}/P \times C_{\text{in}} \times k_H \times k_W}$ for `BoundConv`. Sharding along the output dimension is correct for both layer types since they are linear in their output, and `AllGather` along `dim=0` exactly reconstructs the full matrix.

Hooks `fsdp_gather_node` and `fsdp_free_node` are injected into `backward_general` (CROWN backward) and `IBP_general` (IBP forward): the full matrix is assembled immediately before each layer's computation and freed immediately afterwards. At any instant, at most one full weight matrix resides in GPU memory beyond the shards.

The total implementation is ≈100 lines of Python, versus ≈500 for TP, because FSDP requires no new operator types and does not alter the computational graph.

3.4.2 Bitwise Correctness

FSDP does not change the order or type of any arithmetic operation: each layer receives the full weight matrix assembled via `AllGather` and performs the *identical* floating-point instructions as the single-GPU path. The only difference is that weights are assembled from shards rather than loaded from a contiguous tensor. This is why FSDP achieves $\Delta lb = \Delta ub = 0.00$ (exact zero, not machine epsilon) across all correctness tests (Table 3)—in contrast to TP, which relies on IBP substitution and can deviate by hundreds of units.

3.4.3 Theoretical Memory Analysis

Let the model have d linear layers each of width h, verification uses CROWN, and P GPUs are available. Denote by C the number of A-matrices simultaneously live in GPU memory during the backward pass.

Single GPU: $M_W = dh^2$ (weights) $+ Ch^2$ (A-matrices).

TP: both weights and A-matrices are sharded, $M_{\text{peak}}^{\text{TP}} \approx (d + C)h^2/P$—linear reduction.

FSDP: only weights are sharded, A-matrices remain at full dimension:

$$M_{\text{peak}}^{\text{FSDP}} \approx h^2 \left(\frac{d}{P} + 1 + C \right). \tag{2}$$

The ratio to single-GPU is $(d/P+1+C)/(d+C)$. At d=4, C≈4, P=2: $7/8 = 0.875$, i.e. $\sim$12.5% reduction—matching the observed 34% (baseline overhead from NCCL buffers and metadata are not counted here). Baseline weight storage: $M_W^{\text{FSDP}} = dh^2/P + h^2$, ratio $(1/P + 1/d)$; at d=8, P=2 this gives 0.625, matching the observed 80–90%.

3.4.4 Integration with Complete Verification

FSDP integrates with β-CROWN + BaB without additional algorithmic complexity: each rank runs a standard BaB iteration after receiving full weights via `AllGather`. Branching-constraint variables β are managed identically to single-GPU mode.

During development we discovered a systematic experimental artefact: single-GPU mode used `auto_enlarge_batch_size` and `pruning_in_iteration`, whereas FSDP mode disabled these for `AllGather` synchronisation. The two modes processed *different workloads*, which created a spurious "27×" FSDP advantage in early runs.

To enable fair comparison we introduce `bab.force_synchronous`: a flag that places single-GPU mode on the same code path as FSDP—fixed batch size, no auto-enlarge, no `early_stop`, no `pruning_in_iteration`. Both modes then process an identical number of BaB domains per round, making peak-memory figures directly comparable.

4 Experiments

Setup. Experiments 1–5 ran on Server A: 2× NVIDIA A40 (48 GB VRAM, Ampere, NVLink). Experiments 6–8 ran on Server B: 2× NVIDIA A40 (48 GB, PCIe, no NVLink; `NCCL_P2P_DISABLE=1` was set to prevent P2P hangs). Collective backend: NCCL. Framework: PyTorch 2.x. All distributed runs launched via `torchrun --nproc_per_node=2`.

4.1 Experiment 1: TP Peak-Memory Reduction

Model. A fully connected network with one hidden layer: $f(\mathbf{x}) = W_2 \cdot \text{ReLU}(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$, $W_1 \in \mathbb{R}^{262144\times4096}$ (≈4 GB in float32), $W_2 \in \mathbb{R}^{1\times262144}$. Under TP: W_1 is Column-parallel, W_2 is Row-parallel, following the Megatron-LM pattern [23].

Protocol. CROWN verification, L_∞ robustness, ε=0.01, batch N=2048. Peak memory measured via `torch.cuda.max_memory_allocated`.

Table 1: Peak GPU memory: single-GPU vs. TP=2 (D=4096, H=262144, N=2048, ε=0.01).

Mode	max_alloc (MB)	max_reserved (MB)
Single GPU	26 826	30 874
TP=2, rank 0	13 513	15 512
TP=2, rank 1	13 513	15 512

Reduction factor: $26826/13513 \approx 1.985$, close to the theoretical P=2. The small gap comes from non-shardable structures: input tensor $\mathbf{x} \in \mathbb{R}^{N \times D}$, NCCL buffers, and graph metadata are replicated on every GPU.

4.2 Experiment 2: TP Bound Correctness and Soundness

Protocol. Bounds were computed by single-GPU CROWN (reference) and TP=2 CROWN. Three properties were checked: maximum absolute deviation $|\Delta lb| = |lb_{\text{ref}} - lb_{\text{tp}}|$ and $|\Delta ub|$; soundness ($lb_{\text{tp}} \leq lb_{\text{ref}}$ and $ub_{\text{tp}} \geq ub_{\text{ref}}$ for all outputs); and cross-rank consistency (both ranks produce identical bounds). Models: PyTorch MLPs and VNN-COMP 2022 MNIST-FC ONNX models [3]; ε=0.02, L_∞.

Table 2: TP=2 CROWN bound deviation from single-GPU reference.

Model	Col-Row pairs	$\lvert\Delta lb\rvert$	$\lvert\Delta ub\rvert$	Cross-rank	Sound
PyTorch 256×2	1	3×10^{-8}	3×10^{-8}	0	✓
PyTorch 256×4	2	2.51	2.49	0	✓
ONNX 256×2	1	6×10^{-8}	6×10^{-8}	0	✓
ONNX 256×4	2	5.64	3.98	0	✓
ONNX 256×6	3	792	750	0	✓

Models with one Column–Row pair (no intermediate ReLU in the sharded zone) match the reference up to machine epsilon—confirming that `AllReduce` in the CROWN backward exactly reproduces the full matrix multiplication. With multiple zones, bounds widen due to IBP substitution (Section 3.3.2). Cross-rank deviation is zero in all cases.

4.3 Experiment 3: Automatic ONNX Sharding

`tp_shard_bounded_module` was applied to all three MNIST-FC ONNX models (256×2, 256×4, 256×6) without any manual modification. The function traverses the `BoundedModule` graph in topological order, replaces alternating `BoundLinear` pairs with TP variants, re-executes a forward pass to update graph metadata, and labels sharded zones. All three models pass the soundness checks of Table 2.

4.4 Experiment 4: FSDP Bound Correctness

Protocol. Identical to Experiment 2: single-GPU reference vs. FSDP=2, with checks on $|\Delta lb|$, $|\Delta ub|$, cross-rank deviation, and soundness. Eight test cases: PyTorch MLPs of width h=256 at depth 2, 4, 6 (under both IBP and CROWN) and ONNX 256×2 and 256×4 (CROWN).

All deviations are exactly zero—not machine epsilon but a true zero, reflecting that FSDP changes only *where* weights are stored, not what arithmetic is performed.

Table 3: FSDP=2 bound deviation from single-GPU reference (ε=0.02, MNIST input, L_∞).

Model	Method	$\lvert\Delta lb\rvert$	$\lvert\Delta ub\rvert$	Sound
MLP 256×2	IBP	0.00	0.00	✓
MLP 256×2	CROWN	0.00	0.00	✓
MLP 256×4	IBP	0.00	0.00	✓
MLP 256×4	CROWN	0.00	0.00	✓
MLP 256×6	IBP	0.00	0.00	✓
MLP 256×6	CROWN	0.00	0.00	✓
ONNX 256×2	CROWN	0.00	0.00	✓
ONNX 256×4	CROWN	0.00	0.00	✓

4.5 Experiment 5: FSDP Memory Savings

Protocol. Two metrics: *baseline memory*—allocation after init and sharding, before `compute_bounds` (weight storage cost); *peak memory*— `max_memory_allocated` during `compute_bounds` (includes weights, CROWN A-matrices, intermediate tensors, NCCL buffers).

Tested: MLPs with $h \in \{256, 1024, 4096, 8192\}$ at d=4, and $d \in \{2, 4, 6, 8\}$ at h=4096; CROWN, ε=0.02, input dim 784. FSDP peak is the maximum over both ranks.

Table 4: FSDP=2 vs. single-GPU memory. "—" indicates savings are negligible ($h \leq 1024$: AllGather overhead > weight savings).

Model	Baseline (MB)		Peak (MB)	
	Single	FSDP	Single	FSDP
h=256, d=4	26	11	—	—
h=1024, d=4	187	40	—	—
h=4096, d=4	2 272	417	3 800	2 527
h=8192, d=4	8 760	1 594	14 759	9 782
h=4096, d=8	9 721	930	16 726	10 232

Baseline savings: 57–90%, growing with model size. At $h \geq 4096$ savings stabilise at $\sim$82%, matching the theoretical limit $1/P + 1/d$ (Section 3.4.3). Peak savings are 34–39%: CROWN A-matrices are *not* sharded and are comparable in size to weights at d=4; the weight fraction grows at d=8, improving savings to 39%.

4.6 Experiment 6: FSDP in Complete Verification (BaB)

Motivation. In β-CROWN+BaB, the primary memory consumers are *alpha tensors*: per-neuron ReLU slopes $\alpha_{i,k} \in [0, 1]$ with shape (layers)×(specs)×(batch)×(neurons). For `mnist-net_256x6` ($\sim$1.5 MB weights), alpha tensors occupy $\sim$1.5 GB total—weights are less than 0.1% of peak consumption.

Protocol. `mnist-net_256x6` [3], ε=0.05; exactly 10 BaB rounds; batch 512 and 4 096; `force_synchronous` enabled. Both modes process an identical workload.

Table 5: FSDP=2 vs. single-GPU in β-CROWN+BaB (identical workload).

Config	Batch	Domains	Rounds	Peak (MB)
Single GPU	512	6 236	10	211.7
FSDP=2	512	6 236	10	215.7 / rank
Single GPU	4 096	49 888	10	1 537.3
FSDP=2	4 096	49 888	10	1 541.4 / rank

Sharding 1.5 MB of weights yields no measurable saving; `AllGather` adds $\sim$4 MB overhead. FSDP does not alter bound values or workload— it is a zero-cost correctness guarantee here.

4.7 Experiment 7: FSDP with ViT (VNN-COMP'23)

Model `pgd_2_3_16` (2 Transformer blocks, 3 heads, $\sim$75K parameters, 0.3 MB weights) [4]. Transformers use dynamic reshape operations that `auto_LiRPA`'s JIT tracing "bakes in" at batch size 1, causing shape errors when BaB increases the batch. Three incompatibilities were fixed: (i) `BoundReshape`—first axis replaced by actual batch size when $\prod(\text{new\_shape}) = x.\text{numel}()$, guarding against reshape patterns of the form $[B, N, D] \to [BN, D]$; (ii) `BoundConcat`—JIT-fixed tensor count; (iii) `BoundConstantOfShape`—shape argument updated to current batch.

After the fixes: 78 BaB domains, 10 rounds, peak 1 020.1 MB per rank— bitwise-identical to single-GPU. This is the first confirmed correct execution of a Transformer model in α, β-CROWN with FSDP.

4.8 Experiment 8: FSDP for Convolutional Layers (VNN-COMP'24)

`BoundConv` sharding was extended to convolutional weights $[C_{\text{out}}, C_{\text{in}}, k_H, k_W]$, sharded along dimension 0 (output channels), since convolution is linear in C_{out}. Three files were modified: `fsdp_utils.py` (added `BoundConv` to shardable types, prevented double-sharding), `backward_bound.py` (AllGather/free for `BoundConv` in CROWN backward), `interval_bound.py` (free hooks after IBP propagation).

Validated on CIFAR-100 ResNets [7]. Fair conditions: `force_synchronous`, batch 64, max_iterations 5.

Table 6: FSDP=2 with `BoundConv` sharding on CIFAR-100 ResNet (VNN-COMP'24).

Model	Config	Domains	Rounds	Peak (MB)	$\lvert\Delta lb\rvert$
ResNet-med	Single GPU	130	5	960	—
ResNet-med	FSDP=2	130	5	930 / rank	0.00
ResNet-large	Single GPU	**unsat**	0	4 610.6	—
ResNet-large	FSDP=2	**unsat**	0	4 641.5 / rank	0.00

ResNet-large ($\sim$95 MB weights, $>$ 4 GB alpha tensors): AllGather overhead ($\sim$30 MB) exceeds weight-sharding savings at P=2, confirming that alpha tensors dominate. The **unsat** result—property proved by α-CROWN at initialisation, without BaB—is the first complete verification result obtained under FSDP for a convolutional VNN-COMP model.

Summary. FSDP delivers meaningful peak-memory savings (34–39%) only when weights dominate, i.e. in incomplete (CROWN-only) verification of wide MLPs. In α-CROWN+BaB the bottleneck is alpha tensors; sharding them is the critical next step.

5 Related Work

Bound propagation. CROWN [27] introduced linear relaxation via a backward pass. α-CROWN [28] and β-CROWN [25] extended it with optimisable slopes and dual-variable encoding of branching constraints. BICCOS [22] adds cutting planes from the BaB tree. DeepPoly [24] uses polyhedral abstract interpretation; GPU-accelerated scaling was studied in [19]. OSIP [2] proposes optimised symbolic interval propagation for tighter intermediate bounds.

Complete verifiers. Marabou 2.0 [5] uses the DeepSoI continuous objective. NeuralSAT [12] employs CDCL. nnenum [8] and VeriNet use SIP with dynamic precision switching. MIPVerify [1] encodes ReLU networks as MIPs.

Parameter sharding for training. Megatron-LM [23] introduced intra-layer tensor parallelism for Transformer training. ZeRO [20] shards parameters, gradients, and optimizer states. GPipe [14] introduced pipeline parallelism with micro-batches.

Multi-GPU verification. To our knowledge, no prior work has applied TP or FSDP to neural network verification. Domain-parallel BaB (splitting the search tree across GPUs) has been studied but does not address the per-GPU weight-storage bottleneck.

6 Conclusion

We demonstrate that parameter-sharding strategies transfer from large-scale model training to formal neural network verification with modest engineering effort. TP offers the highest peak-memory reduction ($\approx$2$\times$) but degrades bound tightness for deep networks due to IBP substitution inside sharded zones. FSDP achieves smaller but still meaningful peak savings (34–39% in incomplete verification) with bitwise-exact bounds and seamless integration with complete verification (β-CROWN+BaB).

The central finding is that in α-CROWN+BaB mode, the memory bottleneck is per-neuron *alpha tensors*—not weight matrices. Future work should focus on: (i) sharding alpha tensors across GPUs; (ii) scaling to P=4, 8; (iii) input-split domain-parallel BaB for verification speedup, not just memory reduction.

Acknowledgments

The authors thank the auto_LiRPA / α, β-CROWN team for the open-source codebase that made this work possible.

References

[1] Evaluating robustness of neural networks with mixed integer programming, 2017. URL https://arxiv.org/abs/1711.07356.

[2] Optimized symbolic interval propagation for neural network verification, 2022. URL https://arxiv.org/abs/2212.08567.

[3] The third international verification of neural networks competition (vnn-comp 2022): Summary and results, 2022. URL https://arxiv.org/abs/2212.10376.

[4] The fourth international verification of neural networks competition (vnn-comp 2023): Summary and results, 2023. URL https://arxiv.org/abs/2312.16760.

[5] Marabou 2.0: A versatile formal analyzer of neural networks, 2024. URL https://arxiv.org/html/2401.14461v1.

[6] Robustness verification in neural networks, 2024. URL https://arxiv.org/abs/2403.13441.

[7] The fifth international verification of neural networks competition (vnn-comp 2024): Summary and results, 2024. URL https://arxiv.org/abs/2412.19985.

[8] Stanley Bak. nnenum: Verification of ReLU neural networks with optimized abstraction refinement, 2021. URL https://www.researchgate.net/publication/351683889_nnenum_Verification_of_ReLU_Neural_Networks_with_Optimized_Abstraction_Refinement.

[9] Daniel Bertschinger, Christoph Hertrich, Paul Jungeblut, Till Miltzow, and Simon Weber. Training neural networks is $\exists\mathbb{R}$-complete. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/9813b270ed0288e7c0388f0fd4ec68f5-Paper.pdf.

[10] Stefano Demarchi, Dario Guidotti, Luca Pulina, and Armando Tacchella. VNN-LIB: The verification of neural networks library, 2021. URL https://www.vnnlib.org/.

[11] Stefano Demarchi, Dario Guidotti, Luca Pulina, and Armando Tacchella. Supporting standardization of neural networks verification with vnn-lib and coconet. In *Proceedings of the 6th Workshop on Formal Methods for ML-Enabled Autonomous Systems (FoMLAS 2023)*, volume 16 of *Kalpa Publications in Computing*, pages 47–58, 2023. URL https://easychair.org/publications/paper/Qgdn/open.

[12] Hai Duong, ThanhVu Nguyen, and Matthew Dwyer. NeuralSAT: A DPLL(T)-based framework for verifying deep neural networks, 2023. URL https://github.com/dynaroars/neuralsat.

[13] Dario Guidotti. Enhancing neural networks through formal verification. In *Proceedings of the AI*IA 2019 Doctoral Consortium (AI*IA 2019 DC)*, volume 2495 of *CEUR Workshop Proceedings*, pages 107–112. CEUR-WS.org, 2019. URL https://ceur-ws.org/Vol-2495/paper13.pdf.

[14] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. URL https://arxiv.org/abs/1811.06965.

[15] Jiangchao Liu, Liqian Chen, Antoine Miné, and Ji Wang. Input validation for neural networks via runtime local robustness verification. *CoRR*, abs/2002.03339, 2020. URL https://arxiv.org/abs/2002.03339.

[16] Luca Marzari, Isabella Mastroeni, and Alessandro Farinelli. Advancing neural network verification through hierarchical safety abstract interpretation, 2025. URL https://arxiv.org/abs/2505.05235.

[17] Pierre-Jean Meyer, Alex Devonport, and Murat Arcak. Interval reachability analysis: Bounding trajectories of uncertain systems with boxes for control and verification, 2021. URL https://www.researchgate.net/publication/348624119.

[18] Ramon E. Moore, R. Baker Kearfott, and Michael J. Cloud. *Introduction to Interval Analysis*. SIAM, 2009. ISBN 978-0-898716-69-6.

[19] Christoph Müller, Gagandeep Makarchuk, Gagandeep Singh, Markus Püschel, and Martin Vechev. Scaling polyhedral neural network verification on GPUs, 2021. URL https://files.sri.inf.ethz.ch/website/papers/mller2021neural.pdf.

[20] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models, 2020. URL https://arxiv.org/abs/1910.02054.

[21] Wenjie Ruan, Xiaowei Huang, and Marta Kwiatkowska. Reachability analysis of deep neural networks with provable guarantees. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI)*, 2018. URL https://www.ijcai.org/proceedings/2018/0368.pdf.

[22] Zhouxing Shi, Qirui Jin, J.Z̃ico Kolter, Suman Jana, Cho-Jui Hsieh, and Huan Zhang. Scalable neural network verification with branch-and-bound inferred cutting planes. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/33d93e4dc57453e7667b20f62e7c0681-Paper-Conference.pdf.

[23] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020. URL https://arxiv.org/abs/1909.08053.

[24] Gagandeep Singh, Timon Gehr, Markus Püschel, and Martin Vechev. An abstract domain for certifying neural networks, 2019. URL https://ggndpsngh.github.io/files/DeepPoly.pdf.

[25] Shiqi Wang, Huan Zhang, Kaidi Xu, Xue Lin, Suman Jana, Cho-Jui Hsieh, and J. Zico Kolter. Beta-CROWN: Efficient bound propagation with per-neuron split constraints for neural network robustness verification. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2103.06624.

[26] Kaidi Xu, Zhouxing Shi, Huan Zhang, Yihan Wang, Kai-Wei Chang, Minlie Huang, Bhavya Kailkhura, Xue Lin, and Cho-Jui Hsieh. Automatic perturbation analysis for scalable certified robustness and beyond, 2020. URL https://github.com/Verified-Intelligence/auto_LiRPA.

[27] Huan Zhang, Tsui-Wei Weng, Pin-Yu Chen, Cho-Jui Hsieh, and Luca Daniel. Efficient neural network robustness certification with general activation functions, 2018. URL https://arxiv.org/abs/1811.00866.

[28] Huan Zhang, Shiqi Wang, Kaidi Xu, Linyi Li, Bo Li, Suman Jana, Cho-Jui Hsieh, and J.Žico Kolter. alpha-beta-CROWN: A complete and efficient bound propagation based neural network verifier (vnn-comp 2021–2024), 2023. URL https://github.com/Verified-Intelligence/alpha-beta-CROWN_vnncomp23.